HESTIM — Engineering & Business School

Projet PACTE — 4e année

Année universitaire 2026–2027

AutoMaze AI

Classification d'objets routiers par Machine Learning
pour les véhicules autonomes

Livrable 1 : dossier Go/noGo

Équipe : Rodrigue Nzuzi
[Membre 2]
[Membre 3]

Encadrant : [Nom de l'encadrant]

Date : 4 novembre 2026

Table des matières

1 Introduction **2**

2 Présentation du projet **2**

3 Problématique **3**

4 Objectifs **3**

5 Besoins **4**

6 Cahier des charges **4**

7 Choix technologiques **5**

8 Analyse et conception **6**

9 Méthodologie **8**

10 Planification **8**

11 Points à valider **9**

Demande

Nous sollicitons le Go pour développer un classifieur capable de reconnaître 6 usagers de la route. Le modèle sera entraîné de zéro et testé sur de nouvelles images, puis utilisé dans un simulateur de conduite dans les rues de Casablanca. Il nous reste à obtenir l'avis de notre encadrant sur trois points : le jeu de données, le modèle de référence préentraîné et le périmètre du simulateur (section 11).

1 Introduction

Pour conduire seul, un véhicule doit d'abord comprendre ce qu'il a devant lui. Face à un piéton, il doit s'arrêter ; face à un camion immobile, il vaut mieux le contourner. Reconnaître ce qui se trouve sur la route est donc une étape essentielle de sa perception, et c'est à cette étape que s'intéresse notre projet PACTE de 4e année à HESTIM.

Le sujet nous demande de choisir un jeu de données, de comparer plusieurs modèles de Machine Learning et de comprendre quelles classes leur posent le plus de difficultés. Nous devrons ensuite tester le modèle final sur des images qu'il n'a jamais vues. Ce dossier explique comment nous comptons nous y prendre et prépare la décision de Go/noGo, qui représente 15 % de la note finale.

2 Présentation du projet

AutoMaze AI vise à reconnaître, à partir d'une image, six types d'usagers de la route : les personnes, les vélos, les voitures, les motos, les bus et les camions. Une fois entraîné, le modèle sera testé sur de nouvelles images. Nous l'utiliserons ensuite dans un simulateur, où la voiture adapte sa conduite selon ce que le modèle a reconnu.

Le sujet porte sur la classification, et elle reste au centre de notre travail. Le CNN et le simulateur sont deux ajouts que nous proposons : ils viennent en plus des livrables du descriptif et n'en remplacent aucun.

Nous laissons de côté la détection d'objets dans une image entière, la vidéo et la physique réaliste. Dans le simulateur, la position des obstacles est déjà connue : le modèle n'a donc qu'à dire de quoi il s'agit.

Au 4 novembre 2026, le simulateur est la partie la plus avancée, alors que le travail sur les données et les modèles commence à peine :

Partie	**État**	**Reste à faire**
Environnement Colab et Drive	Notebook de vérification exécuté	Vérification par chaque membre, si ce n'est pas fait
Données	Notebook d'exploration à peine commencé	Choix validé, chiffres par classe, recadrages
Modèles et évaluation	Notebooks préparés, aucun résultat	Tout l'entraînement et l'évaluation
Simulateur	Fonctionne en mode simulation, sans vrai modèle	Branchement de notre modèle et comparaison

Nous sommes trois étudiants, et chacun de nous sera responsable d'un rôle :

Rôle	Responsable	Missions
Données	[Nom]	Exploration, prétraitement, recadrages et découpage
Modèles	[Nom]	Classifieurs classiques, CNN, journal des essais
Évaluation et intégration	[Nom]	Évaluation finale, analyse des erreurs, export du modèle, lien avec le simulateur

3 Problématique

Notre projet part d'une question simple :

> *Un modèle entraîné de zéro sur un jeu de données de taille limitée peut-il reconnaître assez bien les usagers de la route pour guider les décisions d'un véhicule simulé ?*

Pour y répondre, nous devrons aussi éclaircir trois points :

— Parmi les modèles classiques, lequel obtient le meilleur score ? Et un CNN entraîné de zéro fait-il vraiment mieux sur les mêmes données ?

— Quelles classes sont les plus difficiles à reconnaître, et qu'est-ce qui l'explique ?

— Que se passe-t-il concrètement pour le véhicule quand le modèle se trompe, notamment lorsqu'il ne reconnaît pas une personne ?

4 Objectifs

Pour juger nos résultats, nous nous appuierons surtout sur le F1 macro mesuré sur le jeu de test, car il donne le même poids à chaque classe, y compris aux plus rares. Nous préférons fixer la valeur à atteindre après les premiers essais avec les modèles classiques : elle reposera ainsi sur un score réel.

Objectif	Mesure de réussite
Décrire le jeu de données retenu	Provenance, nombre d'images, répartition par classe, résolution et qualité documentées
Comparer SVM, Random Forest et k-NN sur des descripteurs HOG	Accuracy, precision, recall, F1 et matrice de confusion, sur les mêmes données
Entraîner un CNN de zéro	F1 macro comparé au meilleur modèle classique, sur le même jeu de test
Analyser les classes difficiles	Paires de classes confondues et images mal classées expliquées
Tester le modèle final sur de nouvelles images	Le programme affiche par exemple « Image → Voiture (92 %) », où 92 % est un score de confiance
Démontrer le modèle dans le simulateur	Chaque classe prédite déclenche une décision, et les fautes graves sont comptées

5 Besoins

Le projet ne nécessite aucun budget, car tous les outils que nous avons retenus sont gratuits. Ce qui nous manque le plus pour l'instant, c'est la validation du jeu de données par notre encadrant, car toute la suite en dépend.

Besoin	**Détail**	**État**
Données	Images des 6 classes, avec des étiquettes fiables et une licence compatible avec un usage scolaire	Candidat proposé : COCO 2017, à valider
Calcul	GPU pour le CNN, car aucun membre n'en possède	Google Colab (GPU T4 gratuit)
Stockage	Espace partagé pour les données, les poids et les figures	Dossier Google Drive partagé
Poste local	Ordinateur capable d'ouvrir la fenêtre du simulateur, ce que Colab ne permet pas	Disponible
Logiciels	Bibliothèques Python et gestion de versions	Voir la section 7
Équipe	3 membres, avec un responsable par rôle	Noms et rôles à compléter
Encadrement	Validation du jeu de données, du modèle de référence et du périmètre	Voir la section 11

6 Cahier des charges

Les exigences F1 à F6 reprennent ce que demande le descriptif, et F7 correspond à ce que nous ajoutons. Pour chacune, nous avons fixé un critère simple qui permettra de vérifier si elle est remplie.

Exigences fonctionnelles

N°	**Exigence**	**Critère d'acceptation**
F1	Constituer le jeu de données des 6 classes	Source, volume, répartition, résolution et qualité décrits dans le notebook 01
F2	Prétraiter les images	Nettoyage, redimensionnement, normalisation et découpage train / validation / test réalisés
F3	Explorer les données	Nombre d'images par classe, exemples et déséquilibre présentés
F4	Comparer plusieurs classifieurs classiques	SVM, Random Forest et k-NN entraînés sur les mêmes descripteurs HOG
F5	Évaluer les modèles	Accuracy, precision, recall, F1 et matrice de confusion, avec l'analyse des classes difficiles
F6	Prédire sur de nouvelles images	Une image inconnue donne une classe et un score affichés
F7	Entraîner un CNN et l'utiliser dans le simulateur	CNN comparé aux modèles classiques, puis chargé par le simulateur depuis ses métadonnées

Exigences non fonctionnelles

N°	Exigence	Critère d'acceptation
NF1	Entraînement de zéro	Aucun YOLO, détecteur ou poids préentraîné dans notre modèle
NF2	Évaluation fiable	Découpage par image d'origine ; jeu de test utilisé seulement pour l'évaluation finale
NF3	Reproductibilité	Graines fixées, essais notés, notebooks exécutables de bout en bout
NF4	Cohérence du prétraitement	Même taille, même ordre des couleurs et même normalisation dans les notebooks et le simulateur
NF5	Honnêteté des résultats	Aucune métrique inventée ; sans modèle, l'interface affiche « SIMULATION — modèle réel non chargé »
NF6	Licences respectées	Sources des données, OpenStreetMap (ODbL) et illustrations OpenMoji (CC BY-SA 4.0) citées
NF7	Matériel disponible	Entraînement possible sur le GPU gratuit de Colab

Livrables : notebook Python, jeu de données et sa description, comparaison des modèles, rapport, présentation orale et démonstration du classifieur sur de nouvelles images.

7 Choix technologiques

Nous travaillerons en Python sur Google Colab, puisque aucun de nous ne dispose d'un GPU. Seul le simulateur tourne sur nos ordinateurs, car il a besoin d'ouvrir une fenêtre. Pour les données, nous proposons COCO 2017 : parmi les options étudiées, c'est celle qui couvre nos six classes avec de vraies photos de rue et des annotations de référence.

Jeu de données

Option	Classes couvertes	Atouts	Limites
COCO 2017, recadrages par boîte (proposé)	Les 6	Photos réelles, annotations de référence	Découpage à faire, fort déséquilibre attendu, beaucoup de petits objets
CIFAR-10 + CIFAR-100 (plan B)	Les 6, en combinant les deux	Téléchargement simple, images déjà centrées	32 × 32 px seulement, peu routier
Pascal VOC 2012	5 sur 6	Annotations propres, taille raisonnable	Pas de classe camion
Jeux Kaggle « véhicules »	Variable	Prêts à l'emploi	Qualité et licence inégales, souvent sans personne ni vélo

Nous n'avons pas encore les chiffres exacts par classe. Nous les mesurerons dans le notebook 01, et ils nous serviront ensuite à fixer nos seuils de filtrage.

Outils

Besoin	Outil retenu	Pourquoi
Calcul et notebooks	Google Colab, Jupyter, VS Code	GPU gratuit, même environnement pour les 3 membres
Partage	Google Drive pour les données et les poids, GitHub pour le code	Un seul jeu de données ; pas de fichiers lourds dans Git
Données et figures	NumPy, pandas, Matplotlib, seaborn	Exploration et graphiques du rapport
Traitement d'images	OpenCV	Recadrage, redimensionnement et descripteurs HOG
Modèles classiques	scikit-learn	SVM, Random Forest, k-NN et métriques d'évaluation
CNN	PyTorch, torchvision	Architecture sur mesure, entraînée de zéro
Carte du simulateur	OSMnx, OpenStreetMap	Vraies rues de Casablanca
Affichage du simulateur	pygame, Panda3D	Vue 2D, et vue 3D sans GPU dédié
Tests	pytest	Vérifier le simulateur à chaque modification

Les modèles classiques nous donnent le point de comparaison demandé par le sujet. Le CNN nous permettra ensuite de voir si apprendre directement à partir des images apporte un vrai gain, même sans poids préentraînés.

8 Analyse et conception

Notre système se divise en deux parties bien distinctes : d'un côté la chaîne du modèle, qui tourne sur Colab, et de l'autre le simulateur, qui tourne en local. Le simulateur ne réentraîne jamais le modèle : il se contente de charger ses poids et ses métadonnées (figure 1).

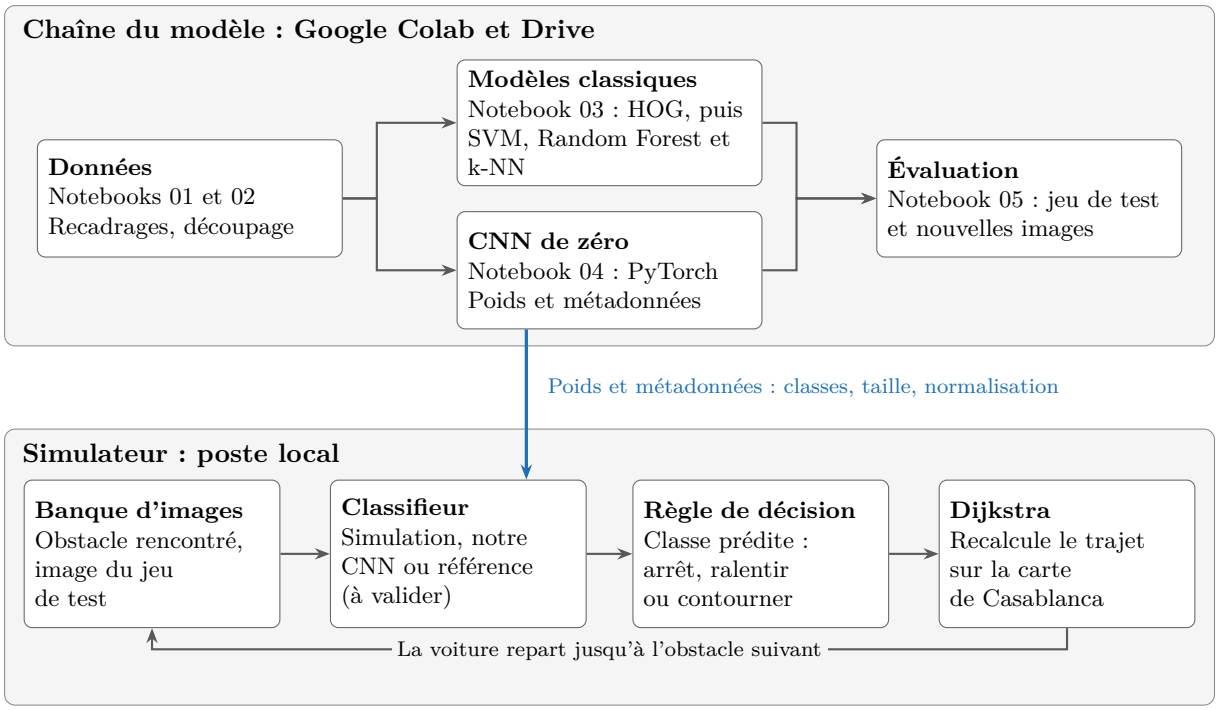


Figure 1 – Architecture : le simulateur ne dépend du modèle que par ses poids et ses métadonnées.

Dans le simulateur, les trois classifieurs possibles fonctionnent de la même façon : ils reçoivent une image et renvoient une classe, un score et 6 probabilités. Nous pouvons donc passer de l'un à l'autre sans toucher à la navigation.

Analyse fonctionnelle. Concrètement, le modèle reçoit le recadrage d'un objet et renvoie une classe avec un score. Si ce score est trop faible, le simulateur affiche « Inconnu » et la voiture s'arrête par prudence. Nous choisirons ce seuil sur le jeu de validation.

Acteurs. L'équipe s'occupe des données, de l'entraînement et de l'évaluation des modèles. L'encadrant valide nos choix au fil du projet, et le jury juge à la fin les résultats et la démonstration.

Règles de décision proposées

Classe prédite	Décision	Effet sur l'itinéraire
Personne	S'arrêter et attendre	Tronçon bloqué quelques pas, puis la voiture repart
Vélo, moto	Ralentir	Coût du tronçon multiplié
Voiture	Contourner	Tronçon bloqué, nouveau calcul
Bus, camion	Contourner largement	Tronçon bloqué, tronçons voisins pénalisés, nouveau calcul
Inconnu	Arrêt prudent	Tronçon bloqué, nouveau calcul

Dans le simulateur, chaque obstacle se comporte selon sa vraie classe : une personne finit de traverser, alors qu'un camion reste immobile. Si le modèle se trompe, la voiture peut donc prendre une mauvaise décision, voire provoquer une collision, et nous pouvons compter ces erreurs. Ces règles sont une première proposition, que nous devons encore valider en équipe.

9 Méthodologie

Nous suivrons les étapes du descriptif, auxquelles nous ajoutons le CNN et le simulateur. Tous les réglages se feront sur le jeu de validation : le jeu de test reste de côté jusqu'à l'évaluation finale.

1. **Acquisition (notebook 01).** Nous récupérons les annotations et les images utiles, et nous justifions le choix du jeu de données.
2. **Exploration (notebook 01).** Nous regardons combien d'images compte chaque classe, si elles sont déséquilibrées et quelle taille font les objets. Nous observons aussi des exemples pour repérer les cas difficiles.
3. **Prétraitement (notebook 02).** Nous filtrons, recadrons, redimensionnons et normalisons les images. Le découpage en train, validation et test se fait par image d'origine, pour que des recadrages d'une même photo ne se retrouvent pas dans deux jeux différents.
4. **Modèles classiques (notebook 03).** Nous extrayons les descripteurs HOG, puis nous comparons SVM, Random Forest et k-NN à un score plancher.
5. **CNN de zéro (notebook 04).** Nous vérifions d'abord que le modèle arrive à mémoriser un petit lot d'images. Ensuite, nous changeons un seul réglage à la fois et nous notons chaque essai.
6. **Évaluation (notebook 05).** Nous mesurons accuracy, precision, recall et F1 sur le jeu de test, puis nous cherchons à comprendre les erreurs à l'aide de la matrice de confusion et des images mal classées.
7. **Application.** Nous testons le modèle final sur de nouvelles images, puis nous le branchons sur le simulateur.

Organisation. Pour éviter les conflits, une seule personne modifie un notebook à la fois. Avant chaque séance, chacun relit le notebook d'un autre membre, et nous notons nos décisions importantes dans un journal de bord.

Principaux risques. Deux risques nous paraissent les plus sérieux. Si la préparation de COCO prend trop de temps, nous passerons à CIFAR et nous discuterons de sa faible résolution dans nos résultats. Si le temps vient à manquer, nous donnerons la priorité au modèle, et le simulateur restera dans sa version actuelle.

Usage de l'IA. Nous avons préparé le simulateur et la structure des notebooks avec l'aide de Claude Code, et la rédaction de ce dossier a aussi été assistée par cet outil. Le choix des données, le prétraitement, l'entraînement et l'analyse des résultats restent notre travail.

10 Planification

Nous avons six semaines devant nous, du 4 novembre au 16 décembre 2026. La priorité immédiate est le jeu de données, car aucun modèle ne peut être entraîné sans lui.

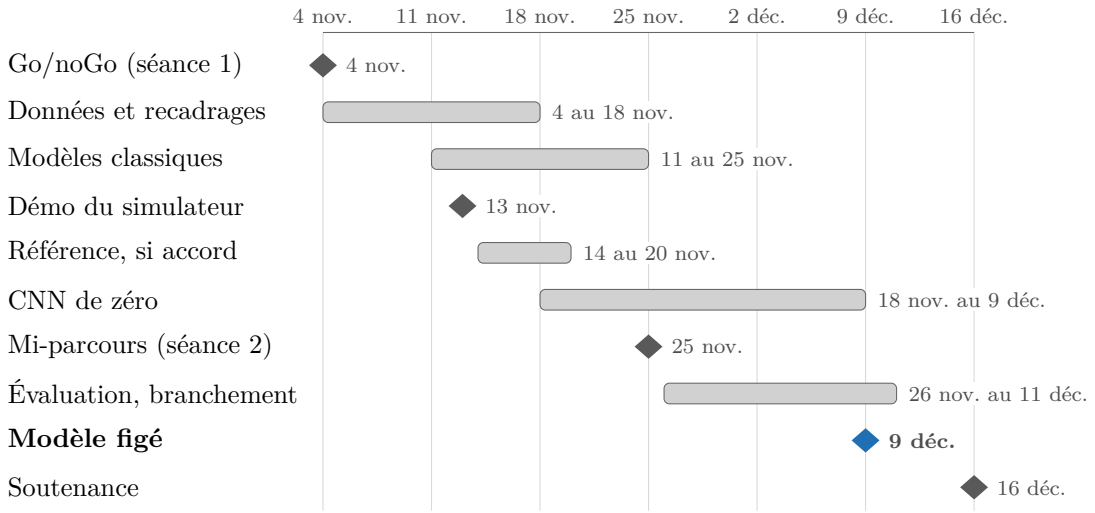


Figure 2 – Calendrier proposé : le modèle est figé le 9 décembre, une semaine avant la soutenance.

Les dates des séances, de la démo du simulateur et du gel du modèle sont fixées. Les durées des travaux sur les données et les modèles sont en revanche une première estimation, que nous ajusterons en équipe. Si une étape prend du retard, ce sont les étapes suivantes qui se décalent, jamais les séances.

11 Points à valider

Pour avancer dans de bonnes conditions, nous avons besoin de l'avis de notre encadrant sur trois points :

- ☐ **Jeu de données :** pensez-vous que COCO 2017 recadré convient, avec CIFAR comme solution de secours ?
- ☐ **Modèle de référence :** pouvons-nous utiliser un ResNet-18 préentraîné, uniquement pour le comparer à notre modèle ? Il resterait bien séparé de notre travail. Si la réponse est non, nous le retirerons, sans que le reste du plan change.
- ☐ **Périmètre :** acceptez-vous le CNN et le simulateur comme ajouts au sujet, sachant que l'évaluation reste centrée sur la classification ?